

ApplyFlow — `tasks.md` (V1)

1. Purpose

This document breaks ApplyFlow V1 into **16 implementation tasks** that can be executed by AI coding agents (Claude / Codex) in a controlled order.

This file exists to solve 4 problems:

1. prevent AI from trying to build the whole app in one shot
2. keep each task small enough to review
3. make progress measurable day by day
4. reduce architecture drift from `spec.md` and `architecture.md`

This task plan assumes:

- the repo currently only contains a root folder named `ApplyFlow/`
 - V1 is a **single-user job application tracker**
 - backend uses **Node.js + Express + MongoDB native driver**
 - frontend uses **React + Vite**
 - one AI can implement and the other AI can review/test
-

2. Ground Rules for All Tasks

2.1 Every task must respect these source documents

Before implementing any task, the AI must follow:

- `spec.md`
- `architecture.md`
- this `tasks.md`

If these documents conflict: 1. `spec.md` defines product behavior 2. `architecture.md` defines implementation structure 3. `tasks.md` defines execution order and scope

2.2 AI must not silently redesign the app

During any task, the AI must **not** introduce architectural changes such as:

- switching MongoDB native driver to Mongoose
- replacing Express with another framework
- introducing Redux/global state for everything
- adding background jobs / cron systems
- adding microservices
- inventing new top-level domains not described in architecture unless explicitly justified

If a change feels necessary, it should be proposed explicitly instead of silently inserted.

2.3 “Done” means more than “code exists”

A task is done only when:

- the code compiles / runs
 - acceptance criteria are satisfied
 - the other AI can review it
 - the code fits the project structure
 - no obvious dead placeholder logic is left pretending to be complete
-

2.4 Clean code requirements apply to every task

All AI-generated code must favor:

- explicit naming
 - thin controllers
 - small service functions
 - readable MongoDB filters
 - minimal magic abstractions
 - no giant catch-all utility files
-

2.5 Task execution workflow

Recommended workflow for each task:

Step A — implement

One AI implements the task.

Step B — review

The other AI reviews:

- correctness
- architecture fit
- edge cases
- naming and readability
- spec compliance

Step C — integrate

You decide whether to:

- accept

- request revisions
 - adjust scope
-

3. Recommended Project Timeline

There are **16 tasks**, but this is **not** a promise that each task equals exactly one day.

Use this as a flexible plan across roughly **14–20 days**:

- some tasks are tiny and can be paired in one day
 - some tasks are heavier and may consume a full day by themselves
 - leave buffer days for bug fixing and spec adjustments
-

4. Task Overview

Phase 0 — Foundation

- **Task 01** — Monorepo scaffold + base folder structure
- **Task 02** — Backend bootstrap: Express, env, Mongo, health check, error middleware
- **Task 03** — Frontend bootstrap: Vite app shell, router, layout skeleton

Phase 1 — Auth

- **Task 04** — Backend auth module
- **Task 05** — Frontend auth flow

Phase 2 — Applications

- **Task 06** — Backend applications create + list
- **Task 07** — Backend applications detail + update + delete
- **Task 08** — Frontend applications list page + create flow

Phase 3 — Events / Timeline

- **Task 09** — Backend events create + list
- **Task 10** — Backend events update + delete + timeline helpers
- **Task 11** — Frontend application detail page + timeline UI

Phase 4 — Dashboard / Intelligence

- **Task 12** — Attention domain rule engine
- **Task 13** — Backend dashboard summary API
- **Task 14** — Frontend dashboard page

Phase 5 — Hardening / Finish

- **Task 15** — Validation + UX consistency + empty states pass
 - **Task 16** — End-to-end review pass, bug fix pass, README/setup polish
-

5. Task Template Legend

Each task below contains:

- **Goal** — what the task is trying to accomplish
 - **Why now** — why it appears in this order
 - **Dependencies** — which earlier tasks must be complete first
 - **Allowed scope** — which files/folders this task may create or edit
 - **Implementation requirements** — concrete instructions
 - **Out of scope** — what the AI must not do in this task
 - **Acceptance criteria** — what must be true before the task is considered done
 - **Manual test checklist** — quick verification steps
 - **Implementation prompt** — prompt for the AI that writes the code
 - **Review prompt** — prompt for the AI that reviews the code
-

6. Task Details

Task 01 — Monorepo scaffold + base folder structure

Goal

Create the initial project structure for both backend and frontend according to `architecture.md`, without implementing business logic yet.

Why now

The repo is currently almost empty. Before any real coding, the project needs a stable structure so future tasks have a clear home.

Dependencies

None.

Allowed scope

Create and edit:

- root-level repo files
- backend/
- frontend/

Implementation requirements

Create the base project structure:

Root

- README.md
- .gitignore
- .env.example

Backend

Create the backend folder structure from `architecture.md`, including:

- src/app.js
- src/server.js
- src/config/
- src/routes/
- src/middlewares/
- src/modules/auth/
- src/modules/application/
- src/modules/event/
- src/modules/dashboard/
- src/domain/attention/
- src/domain/timeline/
- src/db/
- src/utils/
- src/shared/

Add placeholder files where appropriate so the structure is explicit.

Frontend

Create the frontend folder structure from `architecture.md`, including:

- src/app/
- src/api/
- src/pages/
- src/features/auth/
- src/features/applications/
- src/features/events/
- src/features/dashboard/

- `src/components/`
- `src/hooks/`
- `src/utils/`
- `src/constants/`

Package initialization

It is acceptable to initialize:

- backend `package.json`
- frontend Vite app scaffolding

but do **not** implement feature logic yet.

Out of scope

Do **not** implement:

- auth logic
- MongoDB connection logic
- API routes with real behavior
- UI business logic

Acceptance criteria

- The repo has a clear `frontend/` and `backend/` structure.
- Folder names match `architecture.md`.
- Placeholder files do not contain fake business logic.
- The structure is easy to navigate.

Manual test checklist

- Open the repo tree and verify the expected folders exist.
- Confirm there are no random extra top-level folders.
- Confirm no feature logic has been prematurely implemented.

Implementation prompt

Build the initial ApplyFlow repository scaffold based on `architecture.md`. The repo currently only has a root folder `ApplyFlow/`. Create the full backend and frontend folder structure, including placeholder files where appropriate, plus root `.gitignore`, `.env.example`, and README placeholders. Do not implement business logic yet. Keep names explicit and aligned with the architecture document.

Review prompt

Review the scaffolded ApplyFlow project structure against `architecture.md`. Check whether backend and frontend folders are placed correctly, whether module names are consistent, whether any unnecessary architectural inventions were introduced, and whether placeholder files remain

appropriately lightweight. Flag any folder naming drift, missing key directories, or premature feature logic.

Task 02 — Backend bootstrap: Express app, env, Mongo connection, health check, error middleware

Goal

Make the backend actually boot and expose a minimal health route, with centralized environment loading, MongoDB connection setup, and global error handling.

Why now

This is the first “real executable” backend milestone. Later tasks depend on a working server foundation.

Dependencies

- Task 01

Allowed scope

Create or edit:

- backend/package.json
- backend/src/app.js
- backend/src/server.js
- backend/src/config/env.js
- backend/src/config/mongodb.js
- backend/src/routes/index.js
- backend/src/middlewares/error.middleware.js
- backend/src/shared/logger.js
- supporting backend bootstrap files

Implementation requirements

Implement:

- Express app initialization
- JSON middleware
- centralized environment loading
- MongoDB connection bootstrap using native driver
- a health route such as `GET /health`
- global error middleware

- basic not-found route handling if desired

Env support

Expected env values should include at least:

- `PORT`
- `MONGODB_URI`
- `MONGODB_DB_NAME`
- `JWT_SECRET` (can be unused for now but should exist in example env)

MongoDB connection

- use a single shared MongoClient connection
- do not open connections per request
- keep the API simple for later repositories

Out of scope

Do **not** implement:

- auth endpoints
- repositories for business entities
- application/event logic

Acceptance criteria

- Backend starts successfully.
- `GET /health` returns a success payload.
- MongoDB connection bootstrap is centralized.
- Error middleware exists and is wired.

Manual test checklist

- Run backend locally.
- Hit `GET /health`.
- Verify the app starts with env config.
- Verify obvious thrown errors are caught by the global error middleware.

Implementation prompt

Implement the backend foundation for ApplyFlow. Use Node.js + Express + MongoDB native driver. Add centralized env loading, MongoDB connection bootstrap, Express app/server setup, a `GET /health` route, and a global error middleware. Keep the code aligned with `architecture.md`: thin app bootstrap, clean separation of config, routes, and middleware. Do not implement auth or business features yet.

Review prompt

Review the backend bootstrap implementation for ApplyFlow. Check Express setup, env loading, MongoDB connection management, route wiring, and error middleware. Ensure MongoDB is connected centrally, not per request. Verify the structure matches `architecture.md` and that no auth or feature logic was prematurely added.

Task 03 — Frontend bootstrap: Vite app shell, router, layout skeleton

Goal

Make the frontend runnable with route scaffolding and a minimal layout shell.

Why now

Before feature UI work starts, the frontend needs routing and page placeholders.

Dependencies

- Task 01

Allowed scope

Create or edit:

- `frontend/package.json`
- `frontend/src/main.jsx`
- `frontend/src/App.jsx`
- `frontend/src/app/router.jsx`
- `frontend/src/app/providers.jsx`
- route-level page placeholders
- shared layout shell components

Implementation requirements

Set up:

- Vite React app if not already created
- React Router
- placeholder pages:
 - Login
 - Register
 - Dashboard
 - Applications
 - Application Detail

- minimal layout shell for authenticated pages
- minimal navigation structure if useful

The UI can stay visually simple.

Out of scope

Do **not** implement:

- auth logic
- real API integration
- application list rendering
- event timeline rendering

Acceptance criteria

- Frontend starts successfully.
- Navigation between placeholder pages works.
- The structure follows `architecture.md`.

Manual test checklist

- Run frontend locally.
- Visit the placeholder routes.
- Confirm there is a working route structure and app shell.

Implementation prompt

Set up the ApplyFlow frontend foundation using React + Vite. Add React Router, app shell structure, placeholder route pages (Login, Register, Dashboard, Applications, Application Detail), and a minimal layout skeleton aligned with `architecture.md`. Keep it simple and readable; do not implement real business features yet.

Review prompt

Review the frontend scaffold for ApplyFlow. Check router structure, page placement, layout shell organization, and whether the project matches the architecture document. Ensure no business logic has been added prematurely and that the route structure is clean and predictable.

Task 04 — Backend auth module: register / login / me

Goal

Implement backend authentication for:

- register
- login
- current-user retrieval

Why now

Most later routes are user-scoped and require auth middleware.

Dependencies

- Task 02

Allowed scope

Create or edit:

- backend/src/modules/auth/*
- backend/src/routes/auth.route.js
- backend/src/routes/index.js
- backend/src/middlewares/auth.middleware.js
- shared error/helper files if needed

Implementation requirements

Implement:

- POST /auth/register
- POST /auth/login
- GET /auth/me

Register

- create user with email + password
- hash password with bcrypt
- prevent duplicate email

Login

- validate email/password
- return JWT token

Me

- protected route
- return safe user profile for current token

Structure requirements

- controller handles HTTP only
- service handles auth logic
- repository handles user queries
- validator validates auth payloads
- mapper removes sensitive fields from response

Out of scope

Do **not** implement:

- password reset
- refresh tokens
- OAuth
- profile editing

Acceptance criteria

- User can register.
- User can log in and receive a token.
- Auth middleware can decode and verify the token.
- `/auth/me` returns the authenticated user.

Manual test checklist

- Register a user.
- Log in with the same user.
- Use token to call `/auth/me`.
- Verify password hash is not returned.

Implementation prompt

Implement the backend auth module for ApplyFlow using the architecture in `architecture.md`. Add register, login, and me endpoints with Express + MongoDB native driver. Use bcrypt for password hashing and JWT for authentication. Keep controllers thin, services responsible for auth logic, repositories responsible for Mongo access, validators for request validation, and mappers for safe response shaping.

Review prompt

Review the ApplyFlow backend auth module. Verify route behavior for register/login/me, password hashing, duplicate email handling, JWT verification, middleware correctness, repository/service separation, and safe user response mapping. Check for architecture drift, especially controllers doing too much or Mongo logic leaking out of repositories.

Task 05 — Frontend auth flow

Goal

Implement the frontend authentication flow so the app can log in, register, persist auth state, and protect authenticated pages.

Why now

The rest of the app depends on having an authenticated user session.

Dependencies

- Task 03
- Task 04

Allowed scope

Create or edit:

- `frontend/src/api/auth.api.js`
- `frontend/src/features/auth/*`
- login/register pages
- router/protected-route logic
- auth storage helpers if needed

Implementation requirements

Implement:

- register form
- login form
- token persistence
- auth state/store/context
- `/auth/me` bootstrap on app load if token exists
- protected routes for authenticated pages
- logout behavior

Keep the implementation lightweight. A small context/store is enough.

Out of scope

Do **not** implement:

- profile settings
- remember-me complexity
- password reset flows

Acceptance criteria

- User can register from the UI.
- User can log in from the UI.
- Protected pages redirect unauthenticated users.
- Auth state survives page refresh if token exists.

Manual test checklist

- Register a new user from the UI.
- Log in.
- Refresh page and verify session persists.
- Remove token and verify protected route access is blocked.

Implementation prompt

Implement the ApplyFlow frontend auth flow using the existing backend auth API. Add login/register forms, an auth state solution (simple context/store is enough), token persistence, `/auth/me` bootstrap, protected routes, and logout. Keep the structure aligned with `architecture.md` and avoid overengineering the state layer.

Review prompt

Review the ApplyFlow frontend auth flow. Verify login/register behavior, token persistence, protected routes, `/auth/me` bootstrap, and logout handling. Check whether auth logic is reasonably isolated under the auth feature and whether the implementation avoids unnecessary global complexity.

Task 06 — Backend applications: create + list

Goal

Implement the first half of the application domain:

- create application
- list applications

Why now

Applications are the central domain object of the product. Event and dashboard logic depend on them.

Dependencies

- Task 04

Allowed scope

Create or edit:

- `backend/src/modules/application/*`
- `backend/src/routes/application.route.js`
- route registration
- shared constants/utilities if truly needed

Implementation requirements

Implement:

- `POST /applications`
- `GET /applications`

Create application

Must support the fields defined in `spec.md` V1.

List applications

Should return only applications belonging to the current user.

Include the list-query capabilities that were agreed in the spec for V1. If search/filter/sort were defined there, implement them here in a clean and readable way.

Architectural expectations

- controller parses request and calls service
- service coordinates business logic
- repository handles Mongo queries
- validator validates create/list inputs where appropriate

Out of scope

Do **not** implement:

- application detail endpoint
- update/delete
- event routes
- dashboard summary logic

Acceptance criteria

- Authenticated user can create an application.
- Authenticated user can list only their own applications.
- Basic list filtering/search/sorting from the V1 spec works if defined there.

Manual test checklist

- Create multiple applications for one user.
- Confirm they appear in list.
- Confirm a second user cannot see the first user's applications.
- Test any search/filter params defined in the spec.

Implementation prompt

Implement the backend application module for ApplyFlow, but only the create and list capabilities for now. Add authenticated `POST /applications` and `GET /applications` endpoints. Follow `spec.md` for the application fields and list behavior, and follow `architecture.md` for controller/service/repository separation. Keep MongoDB query construction readable and user-scoped.

Review prompt

Review the ApplyFlow backend application create/list implementation. Check field handling against the spec, user scoping, validation, controller/service/repository separation, and the readability of list query logic. Verify that only the authenticated user's applications are returned and that no unrelated application/event logic leaked into this task.

Task 07 — Backend applications: detail + update + delete

Goal

Finish the backend application CRUD by adding:

- application detail
- update
- delete

Why now

Application detail is needed before building the event timeline UI properly.

Dependencies

- Task 06

Allowed scope

Edit:

- application module files
- application routes

- any small shared helper needed for ownership checks or ObjectId handling

Implementation requirements

Implement:

- `GET /applications/:applicationId`
- `PATCH /applications/:applicationId`
- `DELETE /applications/:applicationId`

Rules

- all operations must be user-scoped
- update should allow only fields intended by the spec
- delete should remove the application
- event cascade delete may be implemented here if the event repository already exists; if not, clearly prepare the service structure for it and finish the actual cascade once event repository exists

Because events are not implemented yet at this point, it is acceptable to leave the actual child-event deletion for the later task, **but the code should be written with that future responsibility in mind.**

Out of scope

Do **not** implement event CRUD here.

Acceptance criteria

- Authenticated user can fetch one application by ID.
- Authenticated user can update their own application.
- Authenticated user can delete their own application.
- Cross-user access is blocked.

Manual test checklist

- Create an application, fetch it by ID.
- Update a few fields.
- Delete it.
- Confirm another user cannot access or modify it.

Implementation prompt

Extend the ApplyFlow backend application module with detail, update, and delete endpoints. Add authenticated `GET /applications/:applicationId`, `PATCH /applications/:applicationId`, and `DELETE /applications/:applicationId`. Enforce ownership checks and keep the controller/service/repository separation clean. If event cascade delete cannot be fully implemented yet because the event module does not exist, structure the service clearly so that it can be added cleanly later.

Review prompt

Review the ApplyFlow backend application detail/update/delete implementation. Check ownership enforcement, update field safety, delete behavior, architecture alignment, and whether the code is clearly structured for future event cascade delete without hacky coupling.

Task 08 — Frontend applications list page + create flow

Goal

Build the main applications page so the user can see and create tracked applications from the UI.

Why now

This is the first place where the product starts feeling real from the user's perspective.

Dependencies

- Task 05
- Task 06
- ideally Task 07, though the page can begin before detail/delete are fully wired

Allowed scope

Create or edit:

- `frontend/src/api/application.api.js`
- `frontend/src/features/applications/*`
- `frontend/src/pages/ApplicationsPage/*`
- shared UI components if needed

Implementation requirements

Implement:

- fetch and render application list
- create application form or modal
- status badge display
- basic empty state
- basic loading/error state
- optional search/filter UI if already supported by backend

If delete is already available from Task 07, include delete action in the list if it fits the page cleanly.

Out of scope

Do **not** build the application detail timeline page yet.

Acceptance criteria

- User can open Applications page and see their applications.
- User can create a new application from the UI.
- Basic status display works.
- Empty/loading/error states exist.

Manual test checklist

- Open Applications page with no data.
- Create an application.
- Refresh and confirm it persists.
- If list filters exist, try them.

Implementation prompt

Implement the ApplyFlow frontend applications list page and create flow. Add API integration for listing and creating applications, render the list in a clean UI, include a create form/modal, and show reasonable empty/loading/error states. Keep the feature organized under the applications feature folder and do not yet build the full application detail timeline experience.

Review prompt

Review the ApplyFlow frontend applications list/create implementation. Check API integration, feature structure, state handling, empty/loading/error states, and whether the page feels coherent without overengineering. Ensure application-specific UI lives under the applications feature and that the code is readable.

Task 09 — Backend event module: create + list application events

Goal

Implement the first half of the event domain:

- create event for an application
- list events for an application

Why now

Timeline events are a core part of the tracker and are required for attention flags later.

Dependencies

- Task 07

Allowed scope

Create or edit:

- `backend/src/modules/event/*`
- `backend/src/routes/event.route.js`
- route registration
- event-related repository/service/controller files
- application service/repository only if needed for ownership checks

Implementation requirements

Implement:

- `POST /applications/:applicationId/events`
- `GET /applications/:applicationId/events`

Requirements

- verify the parent application belongs to the authenticated user
- support the event types and fields defined in `spec.md`
- events should be stored in a separate collection
- listing should return events for that application only

Keep event ownership checks explicit and readable.

Out of scope

Do **not** implement update/delete event yet. Do **not** implement dashboard logic here.

Acceptance criteria

- User can add events to their own application.
- User can list events for their own application.
- Another user cannot add or list events for someone else's application.

Manual test checklist

- Create an application.
- Add multiple events to it.
- List the events.
- Verify another user cannot access them.

Implementation prompt

Implement the backend event module for ApplyFlow, but only the create and list operations for now. Add authenticated `POST /applications/:applicationId/events` and `GET /applications/:applicationId/events`. Follow the event model from `spec.md`, keep events in their own collection, and enforce parent-application ownership. Use clean controller/service/repository separation.

Review prompt

Review the ApplyFlow backend event create/list implementation. Check parent application ownership enforcement, event field handling, repository/service separation, and whether the event collection logic is clear and user-safe. Ensure dashboard or unrelated logic was not mixed into the event module.

Task 10 — Backend event module: update + delete + timeline ordering helpers

Goal

Finish event CRUD and add shared timeline ordering logic that later tasks can reuse.

Why now

The frontend application detail page needs a complete event management API.

Dependencies

- Task 09

Allowed scope

Edit:

- event module files
- timeline domain helpers
- route registration if needed

Implementation requirements

Implement:

- `PATCH /applications/:applicationId/events/:eventId`
- `DELETE /applications/:applicationId/events/:eventId`

Also implement the timeline helper logic described in `architecture.md`, such as:

- effective event date resolution if applicable
- stable event sorting logic for timeline display

Keep timeline helpers in the domain/shared area rather than duplicating sorting logic inside controllers.

Out of scope

Do **not** implement dashboard summary here. Do **not** implement attention rules yet.

Acceptance criteria

- User can update an event on their own application.
- User can delete an event on their own application.
- Timeline ordering helper logic exists in a clean reusable location.

Manual test checklist

- Update event fields and confirm persistence.
- Delete an event and confirm it disappears.
- Confirm event listing order behaves as expected for the spec's timeline behavior.

Implementation prompt

Extend the ApplyFlow backend event module with update and delete endpoints, and add the shared timeline ordering/effective-date helpers described in `architecture.md`. Keep event ownership checks explicit, controllers thin, and timeline logic reusable rather than duplicated.

Review prompt

Review the ApplyFlow backend event update/delete implementation and timeline helper extraction. Check ownership enforcement, update safety, delete behavior, and whether timeline ordering logic lives in a clean reusable module rather than being buried in controllers or route handlers.

Task 11 — Frontend application detail page + timeline UI

Goal

Build the application detail page so a user can manage one application and its event timeline from the UI.

Why now

This is the main operational screen of the product, where the user records interview and follow-up history.

Dependencies

- Task 07
- Task 09
- Task 10

Allowed scope

Create or edit:

- `frontend/src/api/event.api.js`
- `frontend/src/pages/ApplicationDetailPage/*`
- `frontend/src/features/events/*`
- application detail-related components under applications feature if needed

Implementation requirements

Implement:

- fetch application detail
- fetch application events
- render event timeline
- add event form
- edit event flow
- delete event flow
- basic timeline ordering display

The page should clearly show:

- application metadata
- current status
- timeline history

It does not need to be visually fancy, but it must be usable.

Out of scope

Do **not** implement dashboard UI here.

Acceptance criteria

- User can open an application detail page.
- User can see application info and timeline events.
- User can create, edit, and delete events from that page.

Manual test checklist

- Open a real application detail page.
- Add a new event.
- Edit an event.
- Delete an event.
- Refresh and verify data consistency.

Implementation prompt

Implement the ApplyFlow frontend application detail page and timeline UI. Integrate the application detail endpoint plus the event create/list/update/delete endpoints. The page should show application metadata, current status, and a usable event timeline with add/edit/delete controls. Keep the structure aligned with the applications/events feature split in `architecture.md`.

Review prompt

Review the ApplyFlow frontend application detail and timeline implementation. Check API integration, feature boundaries, event form behavior, timeline rendering, and general usability. Verify that application-detail concerns and event concerns are organized cleanly rather than tangled into one giant page component.

Task 12 — Attention domain: core rule engine

Goal

Implement the reusable backend attention-flag logic that powers the dashboard reminders.

Why now

This is the product's core "intelligence" layer and should be built before the dashboard API consumes it.

Dependencies

- Task 07
- Task 10

Allowed scope

Create or edit:

- `backend/src/domain/attention/*`
- timeline/shared date helpers if truly needed
- minimal tests if the AI chooses to add them

Implementation requirements

Implement the attention domain described in `architecture.md`.

At minimum, add rule support for:

- `NO_RESPONSE_AFTER_APPLY`
- `NO_RESPONSE_AFTER_INTERVIEW`
- `FOLLOW_UP_OVERDUE`

Critical rule constraints

Respect the previously clarified product rule:

`NO_RESPONSE_AFTER_APPLY`

This flag should only be evaluated for applications whose `currentStatus` is appropriate for this type of silence reminder. It must **not** show for statuses where it would be nonsensical, such as withdrawn or rejected. Follow the exact V1 spec/status-gating logic that was already agreed.

`NO_RESPONSE_AFTER_INTERVIEW`

Likewise, if the application has already moved into a terminal or incompatible status, this flag should not keep appearing.

General requirement

Attention logic must be isolated from HTTP and MongoDB concerns:

- no Express request objects
- no DB calls inside rule functions
- rules should accept plain data and return flags or `null`

Out of scope

Do **not** implement the dashboard endpoint here. Do **not** put rule logic in controllers.

Acceptance criteria

- Attention rules exist in a dedicated domain module.
- Rules are readable and testable.
- Status gating prevents absurd reminders for inappropriate statuses.
- Rule outputs are consistent.

Manual test checklist

Manually test scenarios such as:

- applied 16 days ago, still active, no progress → flag should appear

- applied 16 days ago, then withdrawn → flag should not appear
- interviewed, then rejected → interview silence flag should not appear
- follow-up overdue case behaves as defined in the spec

Implementation prompt

Implement the ApplyFlow backend attention domain as described in `architecture.md`. Add dedicated rule logic for `NO_RESPONSE_AFTER_APPLY`, `NO_RESPONSE_AFTER_INTERVIEW`, and `FOLLOW_UP_OVERDUE`. The rules must be readable, isolated from HTTP/DB concerns, and respect the clarified status gating so silence flags do not appear for inappropriate statuses like withdrawn or rejected. Keep the logic clean and unit-testable.

Review prompt

Review the ApplyFlow attention rule engine carefully. Focus on business correctness, especially status gating for silence flags, timeline/event date reasoning, and separation from HTTP/DB concerns. Check whether the rules are understandable enough for a human reviewer to verify, and flag any hidden assumptions or edge cases that could cause incorrect reminders.

Task 13 — Backend dashboard summary API

Goal

Implement the dashboard summary endpoint that aggregates:

- status counts
- upcoming events
- attention flags

Why now

Once the attention engine exists, the backend can finally expose the product's high-value dashboard view.

Dependencies

- Task 06 / 07
- Task 09 / 10
- Task 12

Allowed scope

Create or edit:

- `backend/src/modules/dashboard/*`
- `backend/src/routes/dashboard.route.js`
- route registration

- minimal helper functions in service/domain if necessary

Implementation requirements

Implement:

- `GET /dashboard/summary`

The dashboard service should orchestrate:

- fetching user applications
- fetching relevant user events
- computing status counts
- computing upcoming events
- computing attention flags via the attention domain

Keep the dashboard as an orchestration module, not a dumping ground for random business logic.

Out of scope

Do **not** implement frontend dashboard UI here.

Acceptance criteria

- The dashboard endpoint returns a coherent payload for the authenticated user.
- It includes status summary data.
- It includes upcoming events.
- It includes attention flags derived from the attention domain.

Manual test checklist

- Seed a few applications with different statuses.
- Add events including future interviews/follow-ups.
- Call dashboard summary and inspect:
 - counts
 - upcoming items
 - attention flags

Implementation prompt

Implement the ApplyFlow backend dashboard summary API. Add authenticated `GET /dashboard/summary` and use the application repository, event repository, and attention domain to build a response containing status counts, upcoming events, and attention flags. Keep the dashboard service focused on orchestration rather than burying rule logic there.

Review prompt

Review the ApplyFlow dashboard summary implementation. Check payload correctness, repository usage, orchestration clarity, and whether attention logic is consumed from the domain layer rather than duplicated in the dashboard service. Verify user scoping and overall readability.

Task 14 — Frontend dashboard page

Goal

Build the dashboard page so the user can actually benefit from the summary data:

- status overview
- upcoming events
- attention reminders

Why now

This is the first truly “product value” page: the user opens the app and immediately sees what needs attention.

Dependencies

- Task 13

Allowed scope

Create or edit:

- `frontend/src/api/dashboard.api.js`
- `frontend/src/features/dashboard/*`
- `frontend/src/pages/DashboardPage/*`

Implementation requirements

Implement a dashboard page that consumes the summary API and displays:

- status summary cards
- attention flags list
- upcoming events list

The UI should be clear and functional. It does not need advanced design polish yet.

Important

The dashboard should surface why a reminder exists in a way that is understandable to the user.

Out of scope

Do **not** redesign the whole app visually. Do **not** add analytics or charts unless already in the spec.

Acceptance criteria

- Dashboard loads summary data from backend.
- Status cards render.
- Attention flags render.
- Upcoming events render.
- Empty/loading/error states exist.

Manual test checklist

- Open dashboard with seeded data.
- Confirm counts match applications.
- Confirm reminders make sense.
- Confirm upcoming events appear in the expected order.

Implementation prompt

Implement the ApplyFlow frontend dashboard page using the backend dashboard summary API. Render status summary cards, upcoming events, and attention flags in a clean, readable layout. Keep the code organized under the dashboard feature and include sensible loading, error, and empty states.

Review prompt

Review the ApplyFlow frontend dashboard implementation. Check API integration, component organization, clarity of the reminder display, and whether the page communicates the app's core value. Ensure loading/empty/error states are handled and the UI code is not tangled into one monolithic component.

Task 15 — Validation pass, UX consistency pass, empty states pass

Goal

Run a targeted polish pass over the existing app to improve consistency and reduce obvious rough edges.

Why now

By this point, the product should function end-to-end. This task is for making it less brittle and less prototype-like.

Dependencies

- Tasks 01–14

Allowed scope

Any frontend/backend files that need cleanup, but changes should stay within V1 scope.

Implementation requirements

Perform a cleanup pass focused on:

- missing validation holes
- inconsistent error handling
- weak empty/loading states
- form-level UX rough edges
- response shape inconsistencies
- naming cleanup if any modules became confusing

Examples of valid improvements:

- better validation messages
- confirm-delete behavior
- clearer empty state on dashboard/applications
- cleanup of duplicated helper logic
- making a controller thinner if it became bloated

Out of scope

Do **not** add major new features. Do **not** redesign architecture. Do **not** sneak in V2 ideas.

Acceptance criteria

- Obvious rough edges in the main user flows are reduced.
- Validation and error handling are more consistent.
- Empty/loading/error states exist where needed.
- The app still respects the V1 scope.

Manual test checklist

Retest these flows:

- register/login/logout
- create/edit/delete application
- create/edit/delete event
- dashboard summary visibility
- invalid input cases
- empty-state scenarios

Implementation prompt

Perform a V1 hardening pass on ApplyFlow. Review the existing implementation and improve validation consistency, error handling consistency, empty/loading/error states, and general UX rough edges

across the main flows. Keep changes within V1 scope and avoid adding new features or architectural redesigns.

Review prompt

Review the ApplyFlow hardening pass. Check whether the changes genuinely improve consistency and robustness rather than introducing new complexity. Look for regressions, accidental scope creep, and places where the cleanup may have hidden behavior instead of clarifying it.

Task 16 — End-to-end review pass, bug fixes, README/setup polish

Goal

Finish the V1 repo in a state where another developer can reasonably clone, run, and understand it.

Why now

A project is not really usable if it only works on the original author's machine or if the setup steps are undocumented.

Dependencies

- Tasks 01–15

Allowed scope

Whole repo, but changes should focus on:

- bug fixes
- integration fixes
- documentation
- setup instructions
- minor cleanup

Implementation requirements

Do a final pass focused on:

- fixing integration bugs discovered across the full app
- ensuring application deletion properly handles child events if not fully finalized earlier
- cleaning startup scripts
- writing README instructions for:
 - prerequisites
 - env setup
 - backend run

- frontend run
- MongoDB requirements
- polishing `.env.example`
- optionally documenting core routes and data model summary

Out of scope

Do **not** start V2 work. Do **not** add features just because they “would be nice.”

Acceptance criteria

- A new developer can understand how to run the app from the README.
- Main flows work together end-to-end.
- Known major integration bugs are addressed.
- The repo feels coherent rather than half-finished.

Manual test checklist

Run through the app from scratch:

- register
- login
- create applications
- add timeline events
- inspect dashboard
- delete application with events
- verify protected routes still behave correctly

Implementation prompt

Perform the final V1 finish pass for ApplyFlow. Fix integration bugs, ensure the repo can be run by another developer, polish README/setup instructions, verify environment configuration, and clean up any remaining V1 inconsistencies. Stay within the existing architecture and product scope; this is a stabilization/documentation pass, not a feature-expansion task.

Review prompt

Review the final ApplyFlow V1 finish pass. Focus on end-to-end coherence, setup clarity, integration correctness, and whether the repository is understandable to a new developer. Verify that the README is accurate, that key flows still work together, and that no accidental V2 scope creep was introduced.

7. Suggested Day-by-Day Use

This is only a suggested cadence, not a hard rule.

Example pacing

- **Day 1:** Task 01 + Task 02
 - **Day 2:** Task 03
 - **Day 3:** Task 04
 - **Day 4:** Task 05
 - **Day 5:** Task 06
 - **Day 6:** Task 07
 - **Day 7:** Task 08
 - **Day 8:** Task 09
 - **Day 9:** Task 10
 - **Day 10:** Task 11
 - **Day 11:** Task 12
 - **Day 12:** Task 13
 - **Day 13:** Task 14
 - **Day 14:** Task 15
 - **Day 15:** Task 16
 - **Day 16–20:** buffer for revisions, review disagreements, bug fixes, and polish
-

8. How to Use This With Claude and Codex

Recommended operating model

Option A

- **Claude implements**
- **Codex reviews**

Option B

- **Codex implements**
- **Claude reviews**

Both are fine. The key is consistency.

For each task:

1. Open the relevant task section in this file.
2. Copy the **Implementation prompt** into the coding AI.
3. Let it generate or modify code.
4. Copy the **Review prompt** into the second AI.
5. Compare:
6. spec compliance
7. architecture fit
8. reviewer objections

9. Decide whether to merge or request changes.

9. Final Note

This task plan intentionally optimizes for:

- **incremental delivery**
- **AI reviewability**
- **low ambiguity**
- **clean separation of concerns**
- **a real V1, not a giant half-built system**

If future V2 features are added later (mail parsing, richer reminders, analytics, resume storage, etc.), they should be added through **a new task plan**, not by quietly bloating this V1 schedule.